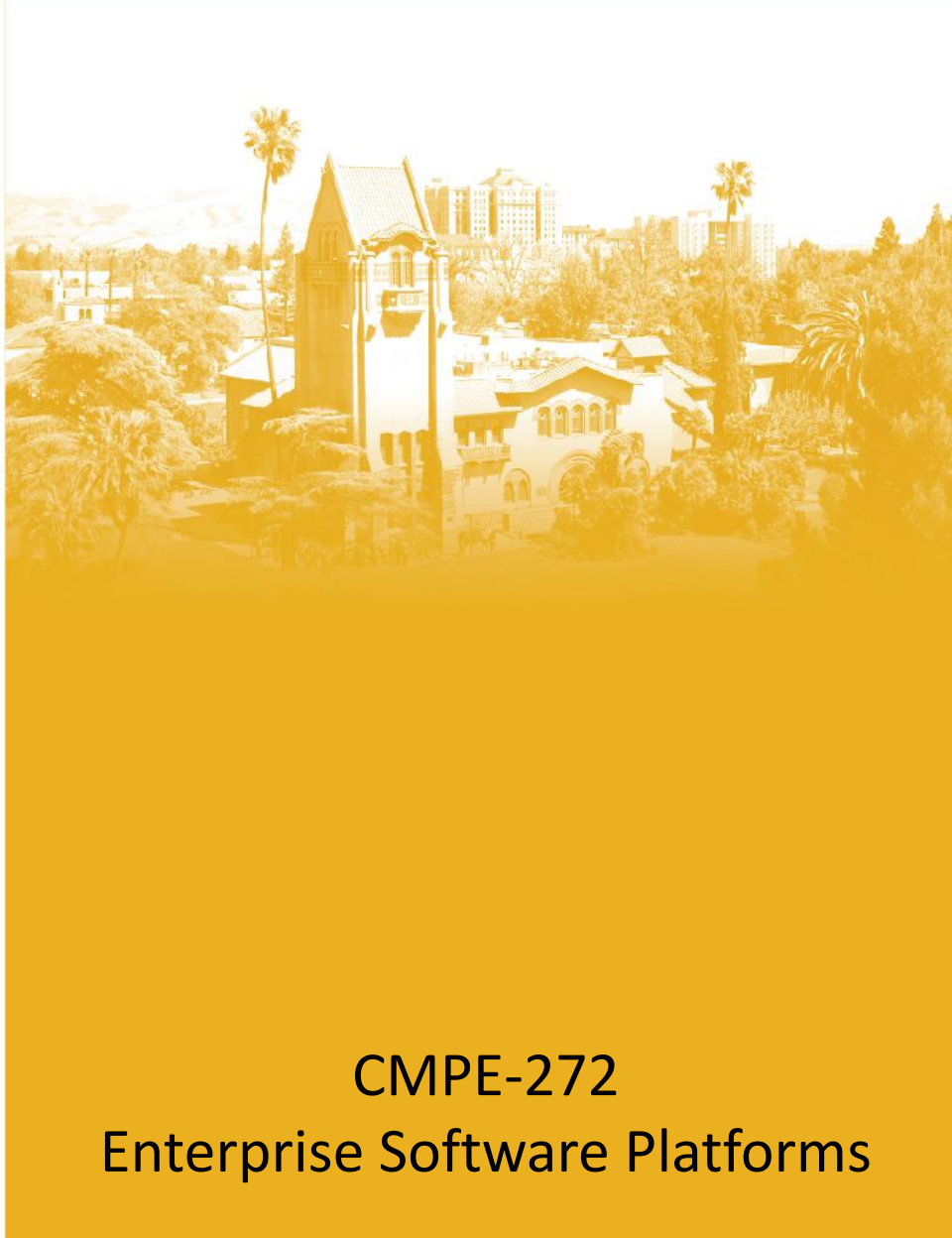




CMPE-272
Enterprise Software Platforms

Enterprise Software Platforms

Instructor: Andrew Bond

IT News

- AWS, September 18: Amazon SNS now accepts messages up to 1 MiB, four times the old 256 KiB limit. SQS made the same change in August 2025. A large message is still a design smell.
- Camunda: version 8.9 shipped April 14 with the Agent2Agent protocol and relational databases as secondary storage. Camunda 7 Community Edition ended in October 2025; the Enterprise Edition is supported to April 2030. Tonight's exercise uses Camunda 8.
- Kafka: Apache Kafka 4.3 (May 22) and 4.3.1 (June 25) are current, and 4.x runs without ZooKeeper. IBM closed its 11 billion dollar purchase of Confluent on March 17. The largest Kafka vendor is now part of IBM.
- Temporal, at its Replay conference in May: worker versioning and task-queue priority reached general availability; serverless workers on AWS Lambda are in pre-release. Code-first workflow engines are becoming managed services.
- Tonight's question: your project has to call three systems and keep working when one of them is down. Which of these four items changes how you would build it?

**UNIT 6 · BUSINESS PROCESSES,
MESSAGING AND SERVICE
COMPOSITION (WEEK 7)**

Tonight

- Part I. Modelling vocabulary: the UML and ER diagrams your project will actually use. About 25 minutes.
- Part II. Business process modelling and BPMN, and what a process engine does with a diagram. About 35 minutes.
- Break at 7:10.
- Part III. Messaging and event-driven integration: queues, topics, delivery guarantees, the outbox pattern. About 35 minutes.
- Part IV. Service composition: orchestration, choreography, sagas, and the engines that run them. About 20 minutes.
- 8:15. Two short in-class exercises, Amazon SQS and Camunda, in your project teams.
- The Project Abstract Submission is due tonight at 11:59 PM, one per team.

What you should be able to do after today

- Read a class, sequence and ER diagram, and draw the one your project needs.
- Read a BPMN diagram: pools, lanes, tasks, events and gateways.
- Say when a queue is right and when a topic is right.
- State what at-least-once delivery forces you to build, and build it.
- Explain why writing to a database and a broker in two steps fails, and what the outbox pattern does about it.
- Choose orchestration or choreography for a flow and defend the choice.
- Design a saga with its compensating actions.
- Place Camunda, Temporal and Step Functions on one map and pick one for a given job.

PART I: MODELLING VOCABULARY

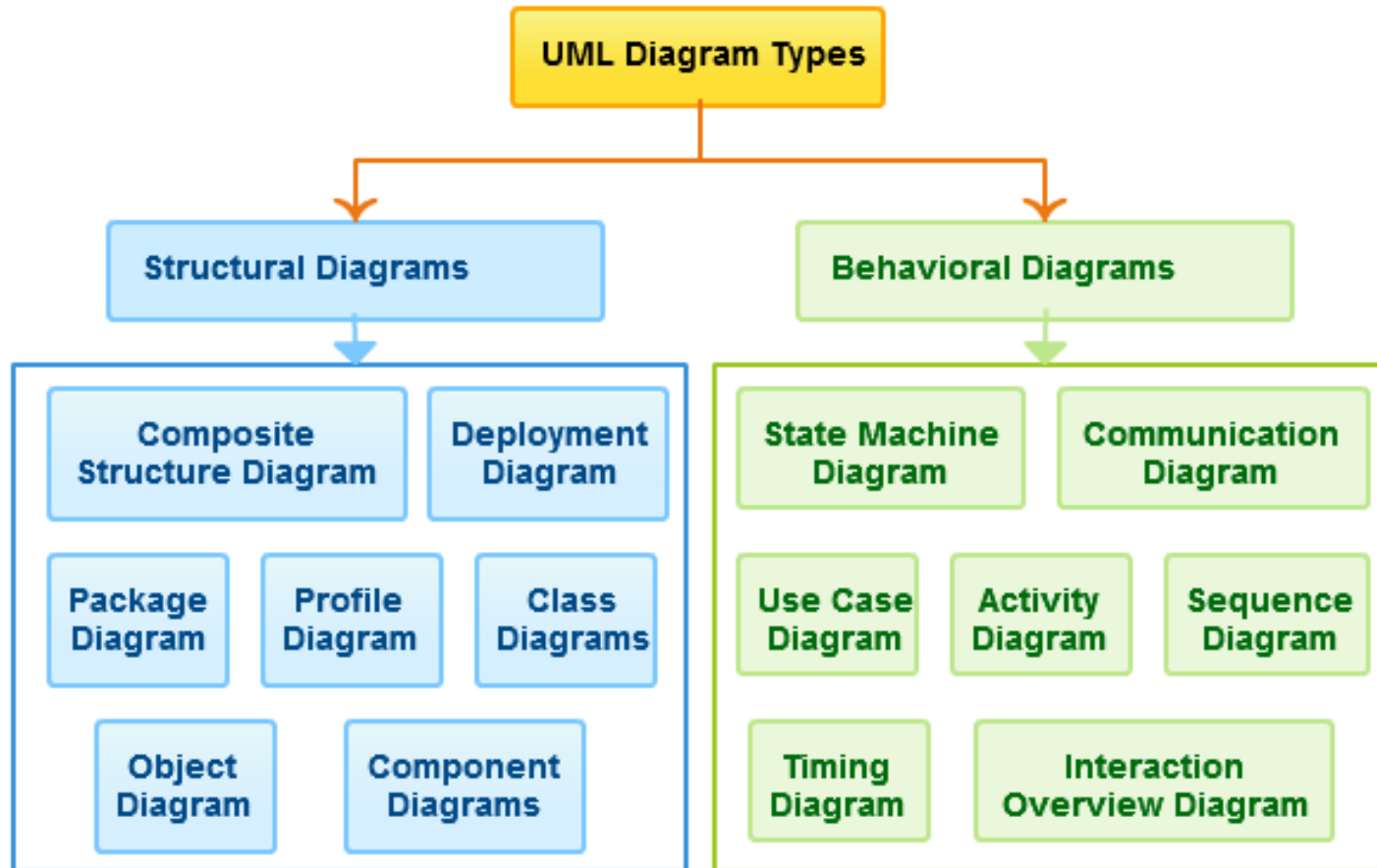
UML and ER, as far as your project needs them

UML – Unified Modelling Language

Unified Modeling Language (UML) is a standardized, general-purpose modeling language in the field of software engineering (for specifying, documenting and supporting development of software systems and visualizing a system's architectural blueprints).

- UML includes a set of graphic notation techniques to create visual models of object-oriented software-intensive systems.
- Core concepts in UML include: **actor, attribute, class, interface, object, activity, event, message, state, use case, association, composition, depends, generalization, inheritance, multiplicity, role**
- UML was developed by Grady Booch, Ivar Jacobson and James Rumbaugh at Rational Software in the 1990s. It was adopted by the Object Management Group (OMG) in 1997
- In 2000 the Unified Modeling Language was accepted by the International Organization for Standardization (ISO) as industry standard for modeling software systems.

UML Diagrams

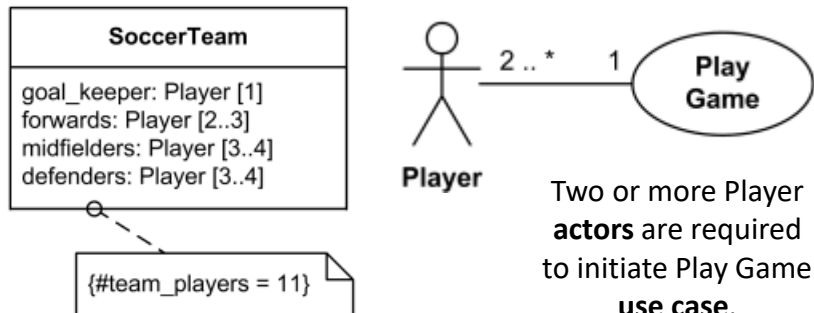


UML Multiplicity

Multiplicity is a definition of **cardinality** - i.e. **number of elements** - of some collection of elements by providing an inclusive interval of non-negative integers to specify the allowable number of instances of described element. Multiplicity interval has some **lower bound** and (possibly infinite) **upper bound**:

multiplicity-range ::= [lower-bound '..'] upper-bound
lower-bound ::= natural-value-specification
upper-bound ::= natural-value-specification | '*'

Lower and upper bounds could be natural constants or constant expressions evaluated to natural (non negative) number. Upper bound could be also specified as asterisk '*' which denotes unlimited number of elements. Upper bound should be greater than or equal to the lower bound.



Multiplicity	Option	Cardinality
0..0	0	Collection must be empty
0..1		No instances or one instance
1..1	1	Exactly one instance
0..*	*	Zero or more instances
1..*		At least one instance
5..5	5	Exactly 5 instances
m..n		At least m but no more than n instances

<https://www.uml-diagrams.org/multiplicity.html>

UML Association

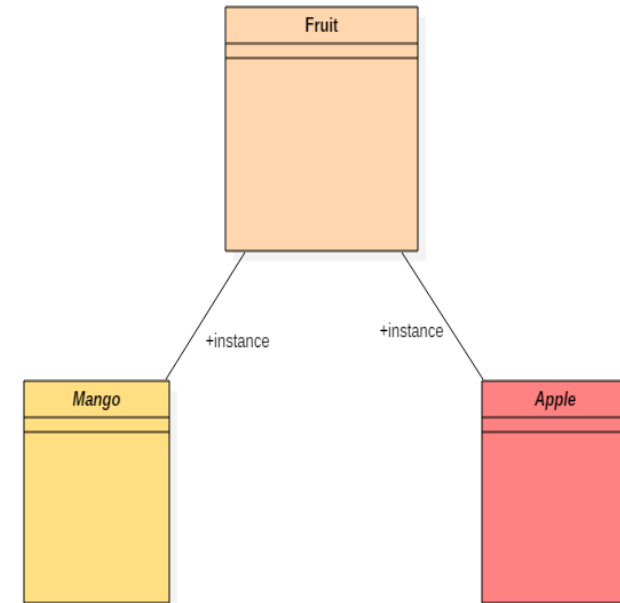
- A structural relationship that represents objects can be connected or associated with another object inside the system

Constraints:

- **{implicit}** – Implicit constraints specify that the relationship is not manifest; it is based upon a concept.
 - **{ordered}** – Ordered constraints specify that the set of objects at one end of an association are in a specific way.
 - **{changeable}** – Changeable constraint specifies that the connection between various objects in the system can be added, removed, and modified as per the requirement.
 - **{addOnly}** – It specifies that the new connections can be added from an object which is situated at the other end an association.
 - **{frozen}** – It specifies that when a link is inserted between two objects, then it cannot be modified while the frozen constraint is active on the given link or a connection.
- We can also create a class that has UML Association properties; it is called as an association class.

Reflexive Association

- The reflexive association is a subtype of association relationship in UML. In a reflexive association, the instances of the same class can be related to each other. An instance of a class is also said to be an object.



Association vs. Aggregation vs. Composition

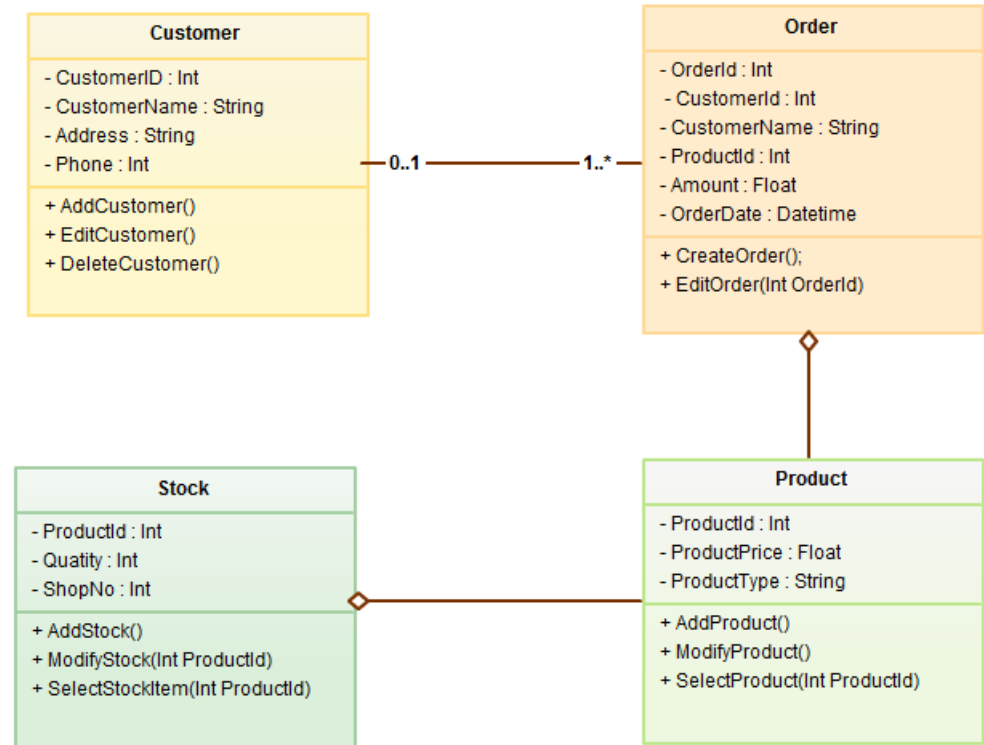
Association	Aggregation	Composition
Association relationship is denoted using an arrow.	Aggregation relationship is denoted using a straight line with an empty arrowhead at one end.	Composition relationship is denoted using a straight line with a filled arrowhead at any one of the ends.
Association can exist between two or more classes in UML.	Aggregation is a part of an association relationship.	The composition is a part of an association relationship.
There can be one-one, one-many, many-one, and many-many association present between the association classes.	Aggregation is considered as a weak type of association.	The composition is considered as a strong type of association.
In an association relationship, one or more objects can be associated with each other.	In an aggregation relationship, objects that are associated with each other can remain in the scope of a system without each other.	In a composition relationship, objects that are associated with each other cannot remain in the scope without each other.
Objects are linked with each other.	Linked objects are not dependent upon the other object.	Objects are highly dependent upon each other.
In UML Association, deleting one element may or may not affect another associated element.	In UML Aggregation, deleting one element does not affect another associated element.	In UML Composition, deleting one element affects another associated element.
Example: <i>A teacher is associated with multiple students.</i> <i>Or</i> <i>A teacher provides instructions to the students.</i>	Example: <i>A car needs a wheel, but it doesn't always require the same wheel.</i> <i>A car can function adequately with another wheel as well.</i>	Example: <i>A file is placed inside the folder.</i> <i>If one deletes the folder, then the file associated with that given folder is also deleted.</i>

Class Diagram

Class diagrams are arguably the most used UML diagram type. It is the main building block of any object oriented solution. It shows the classes in a system, attributes and operations of each class and the relationship between each class.

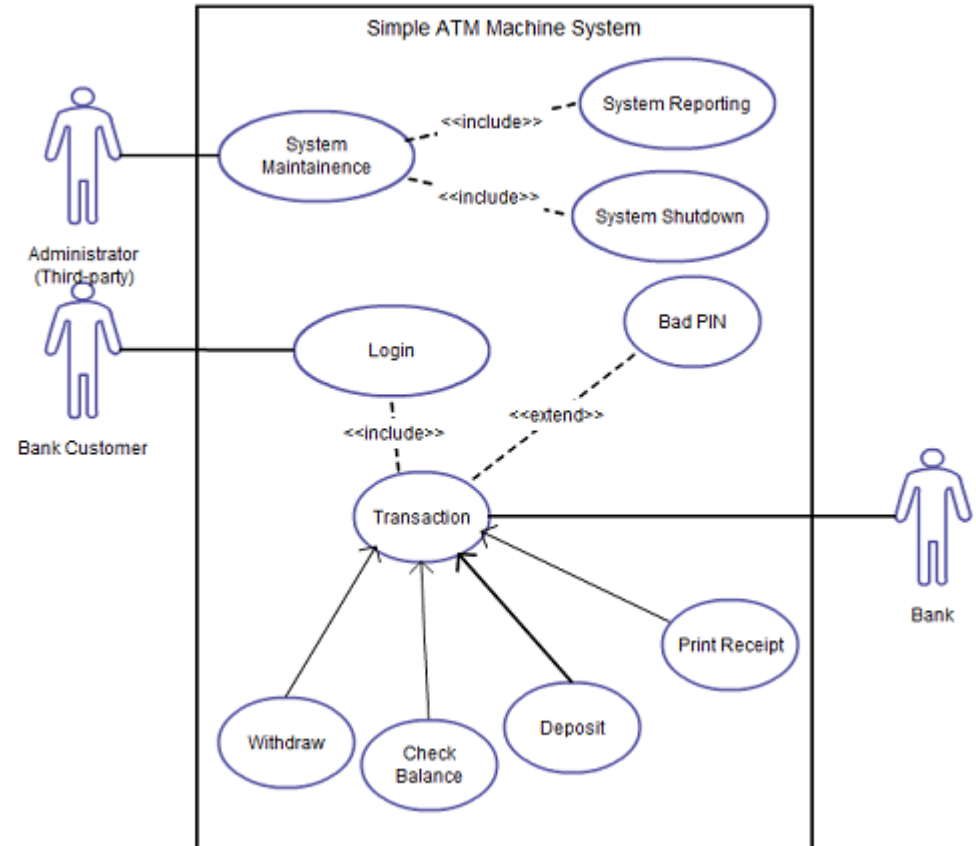
In most modeling tools a class has three parts, name at the top, attributes in the middle and operations or methods at the bottom. In large systems with many related classes, classes are grouped together to create class diagrams. Different relationships between classes are shown by different types of arrows.

Class Diagram for Order Processing System



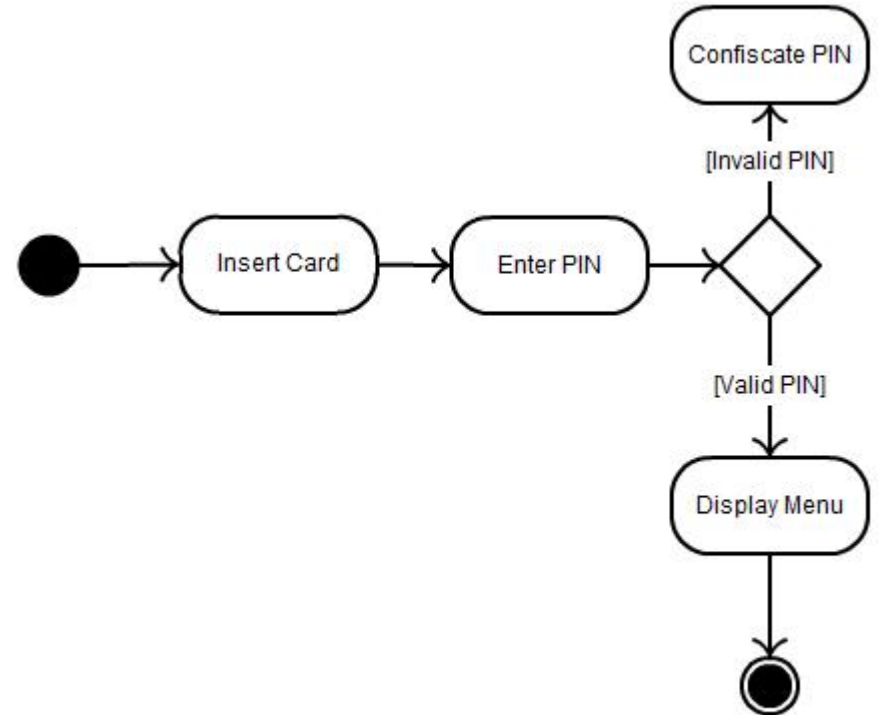
Use Case Diagram

- Use case diagrams gives a graphic overview of the **actors** involved in a system, different functions needed by those actors and how these different functions are interacted
- It's a great starting point for any project discussion because you can easily identify the main **actors** involved and the main processes of the system



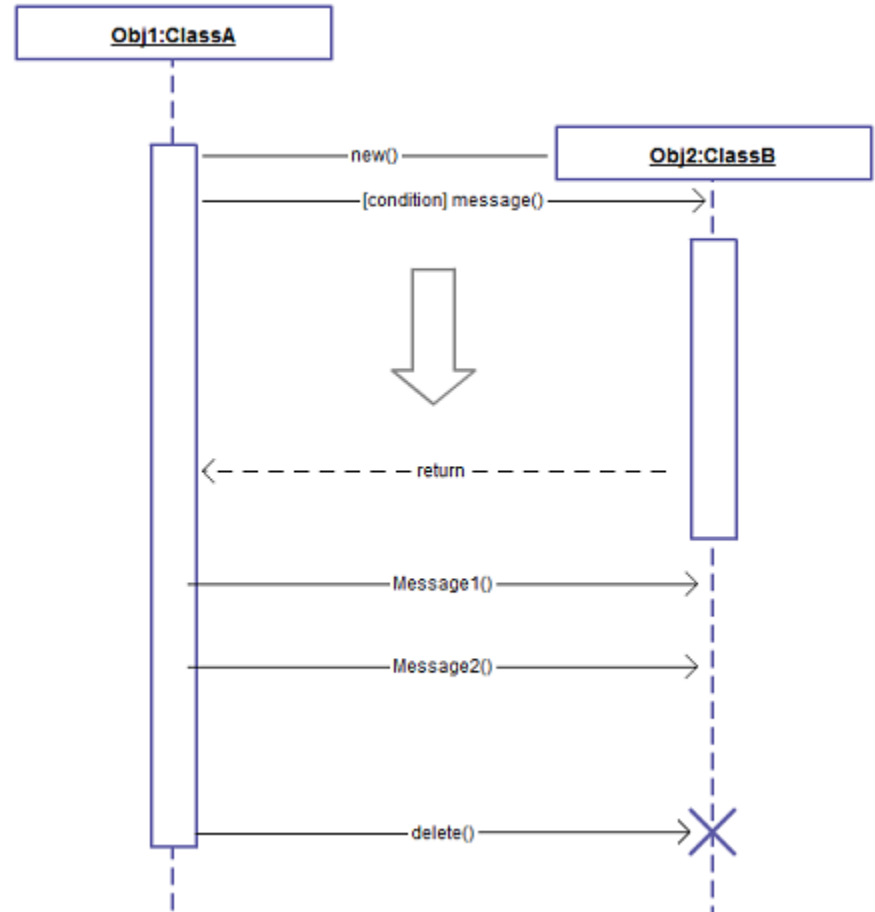
Activity Diagram

- Activity diagrams represent workflows in a graphical way
- They can be used to describe business workflow or the operational workflow of any component in a system
- Sometimes activity diagrams are used as an alternative to State machine diagrams

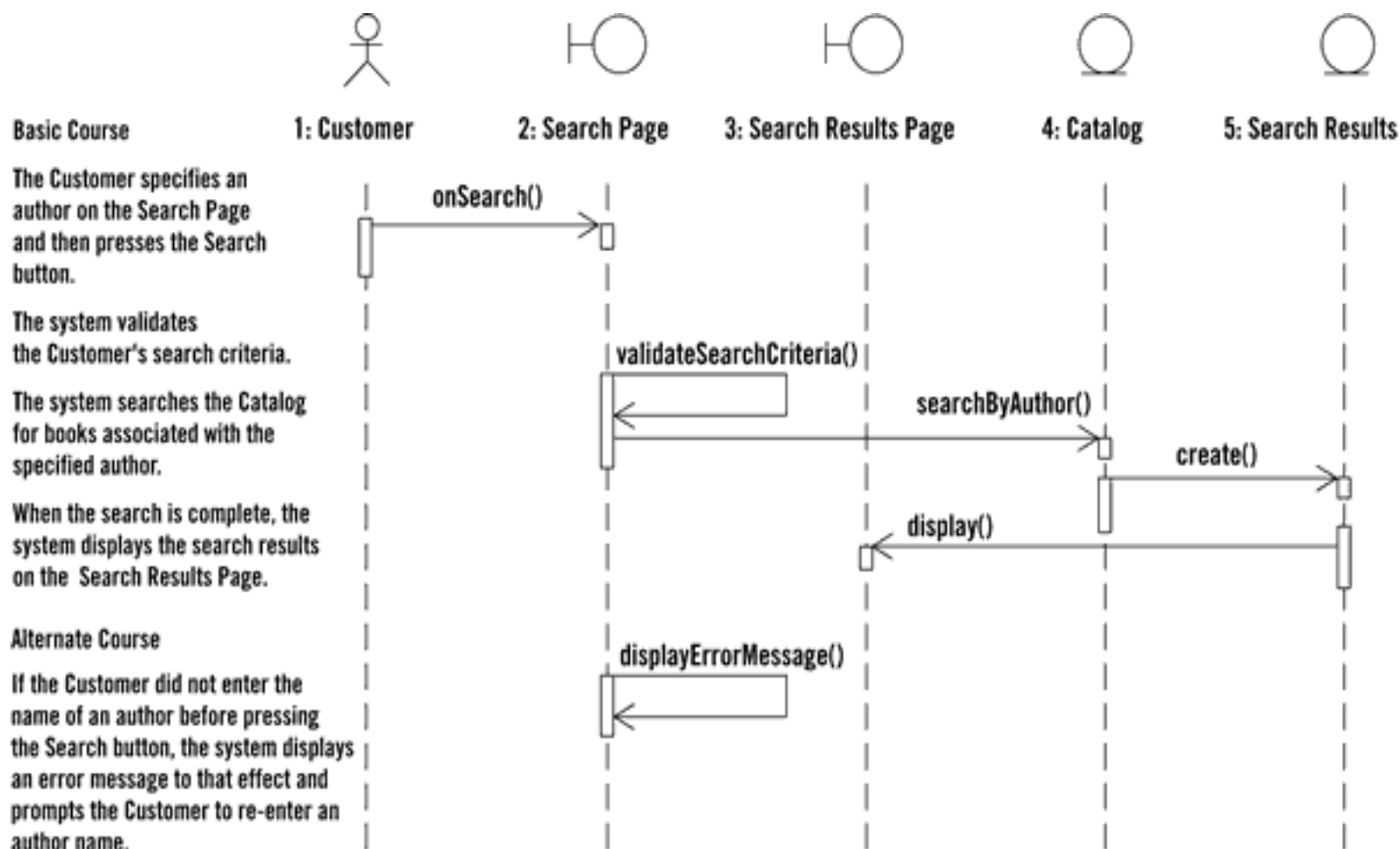


Sequence Diagram

- Sequence diagrams in UML shows how object interact with each other and the order those interactions occur
- It's important to note that they show the interactions for a particular scenario
- The processes are represented vertically and interactions are show as arrows. See also: [Basics of Sequence diagrams](#).



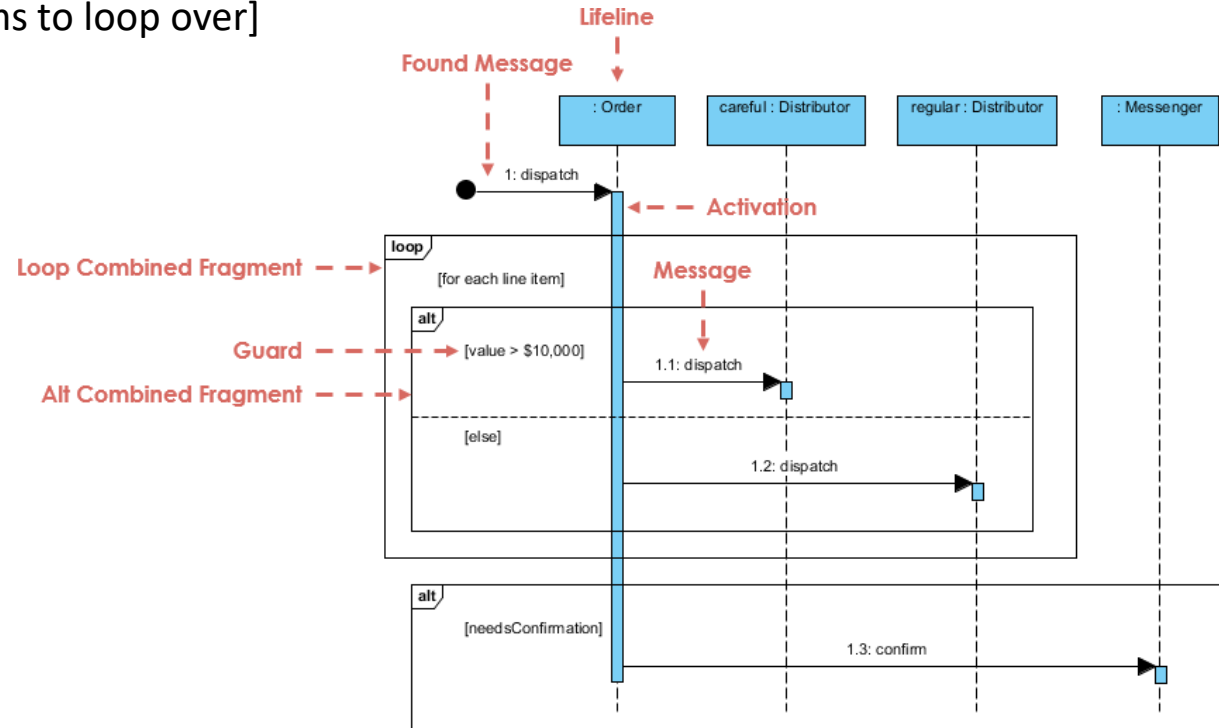
Sequence Diagrams in UML shows the interaction between two processes



Selection and loops

frame: box around part of diagram to indicate if or loop

- if -> (opt) [condition]
- if/else -> (alt) [condition], separated by horizontal dashed line
- loop -> (loop) [condition or items to loop over]



In this example:

- For each of the line item in the order - if the item value > 10,000 then dispatch through careful Distributor, else dispatch through regular distributor.
- If any items fall into "needsConfirmation" condition the confirm message will be sent to the messenger.

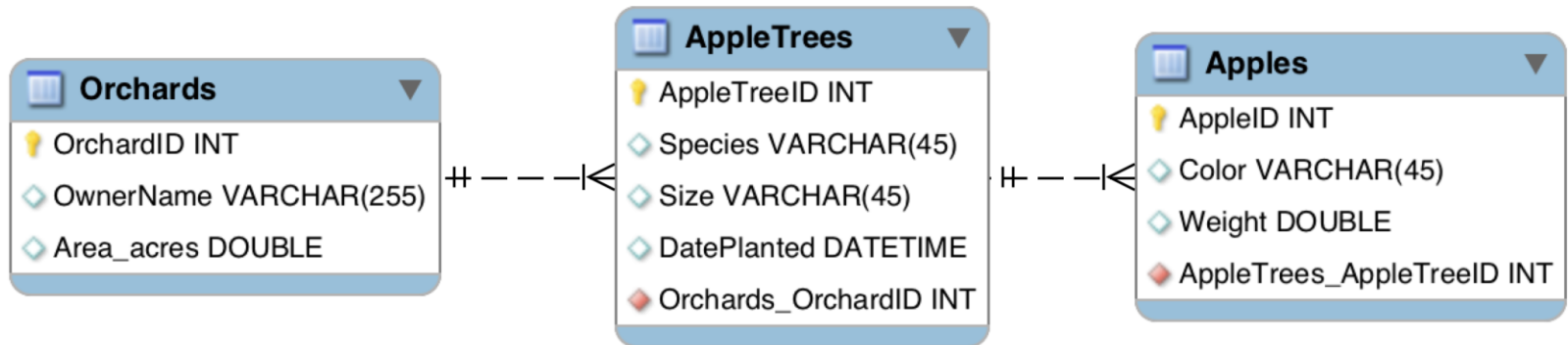
Why not just code it?

- Sequence diagrams can be somewhat close to the code level
- So why not just code up that algorithm rather than drawing it as a sequence diagram?
 - a good sequence diagram is still a bit above the level of the real code (not all code is drawn on diagram)
 - sequence diagrams are language-agnostic (can be implemented in many different languages)
 - non-coders can do sequence diagrams
 - easier to do sequence diagrams as a team
 - can see many objects/classes at a time on same page (visual bandwidth)

ENTITY RELATIONSHIP DIAGRAM

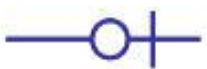
Entity Relationship Diagram

- Documentation of the structure of the data
- Used to communicate the design
- Serve as the basis for data model implementation



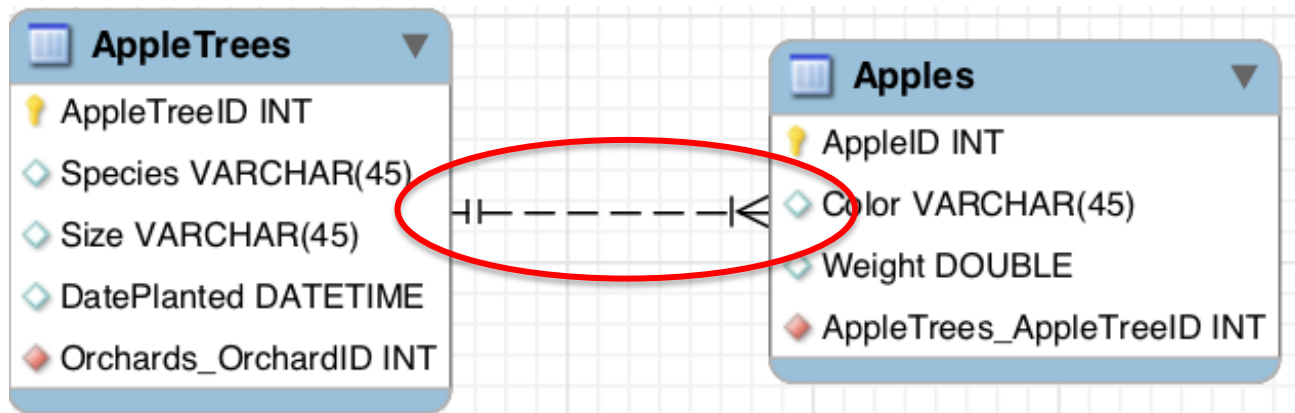
ER Diagram Relationship Notation

- Multiple notation systems are used
- Each software program is a little different
- Most common is “Crows Foot”

	Crows Foot	Alternative
from Zero to Many		0 .. *
from One to Many		1 .. *
from One to One i.e., one and only one		1 .. 1
from Zero to One		0 .. 1

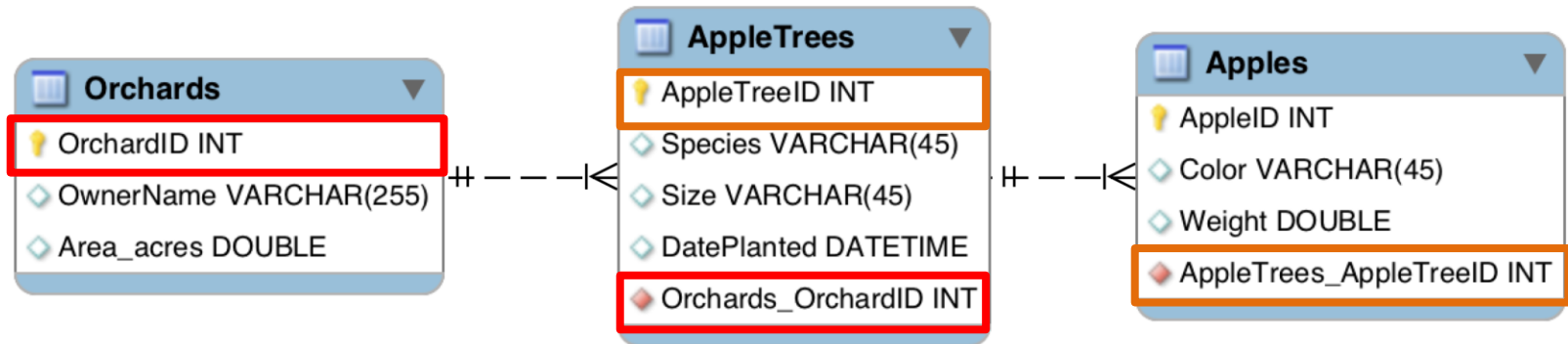
Relationship Cardinality and Participation

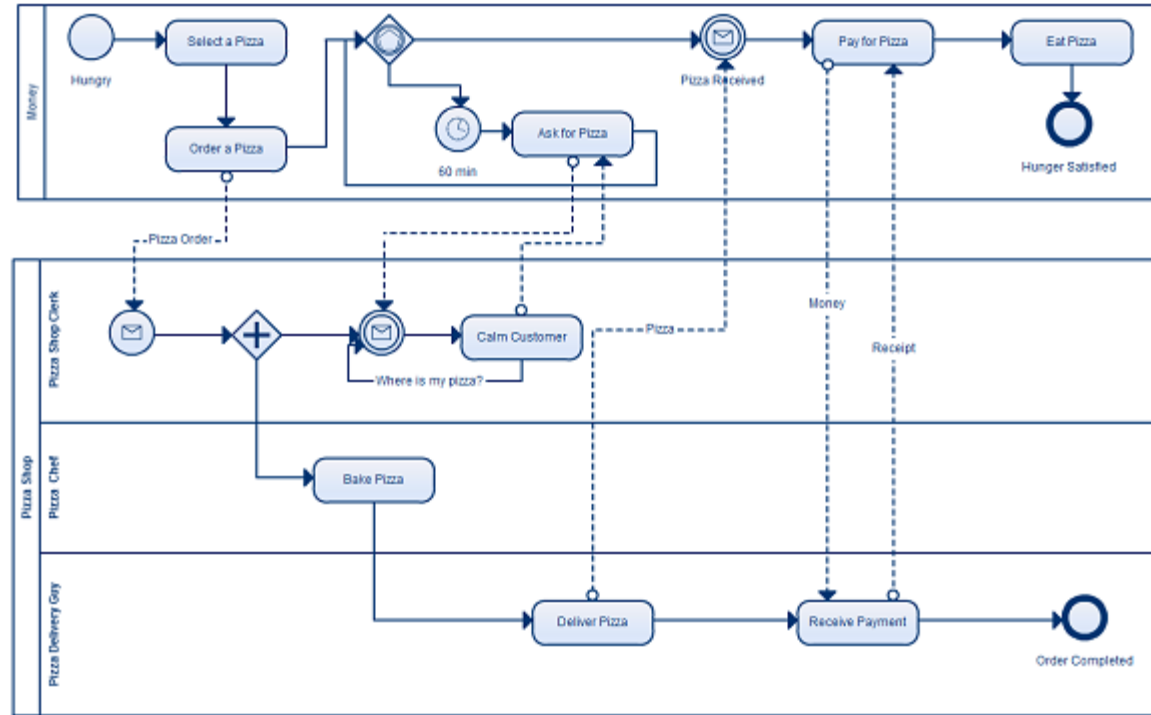
- **Cardinality:** maximum number of times an instance in one entity can be associated with instances in the related entity
- **Participation:** minimum number of times an instance in one entity can be associated with instances in the related entity



Primary and Foreign Keys

- Each row in a table should have an attribute that is a persistent, unique identifier – the “Primary Key”
- Primary key in “parent” table
- Foreign key in “child” table





PART II: BUSINESS PROCESS MODELING AND BPMN

Business Process Modeling (BPM)

- **Business process modeling (BPM)** in business process management and systems engineering is the activity of representing processes of an enterprise, so that the current process may be analyzed, improved and automated
- Modeling and simulation functionality allows for pre-execution “what-if” modeling and simulation. Post-execution optimization is available based on the analysis of actual as-performed metrics.^[7]
- [Use case diagrams](#) created by [Ivar Jacobson](#), 1992 (integrated in [UML](#))
- [Activity diagrams](#) (also adopted by UML)

Business Process Model and Notation (BPMN)

- a standard for business process modeling that provides a graphical notation for specifying business processes in a Business Process Diagram (BPD) based on a flowcharting technique very similar to activity diagrams from Unified Modeling Language (UML)
- BPMN has been maintained by the Object Management Group (OMG) since the two organizations merged in 2005
- Though it is an OMG specification, BPMN is also ratified as ISO 19510.
- The latest version is BPMN 2.0.2, published in January 2014

Modeling Concepts in BPMN



Activity



Event

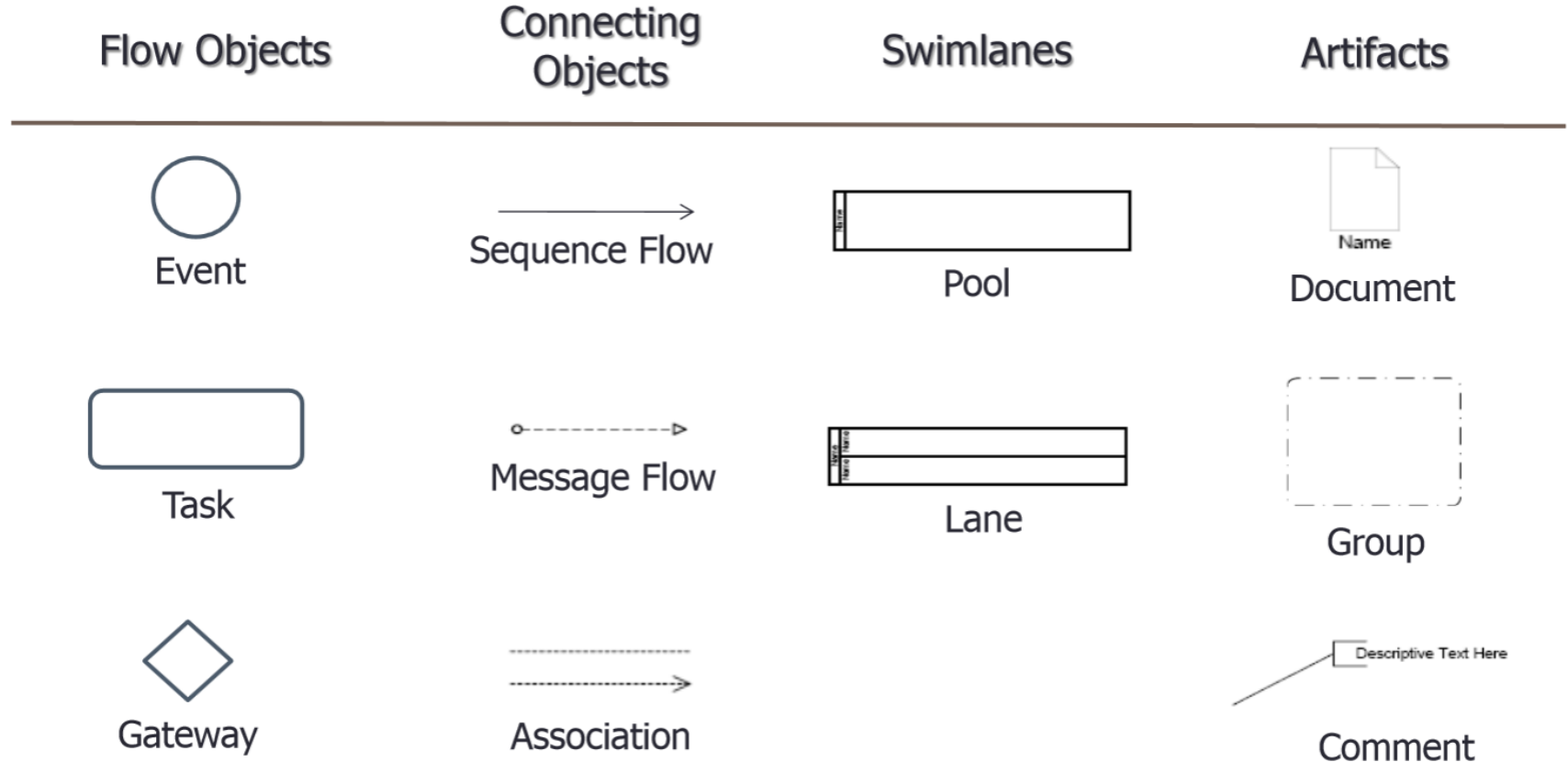


Gateway



Connector

BPMN Symbols Notation per Category



Pools & Lanes

Pools

- A pool is a basic BPMN element that sets the boundaries of a business process. A pool will contain at most one business process. This means that two processes have to be modeled in two different pools. A pool may have visible internal details in the form of a process that will be executed (called a “White-box Pool”), or a pool may have no visible internal details (called a “Black-box Pool”). The type of pool that should be used will depend on the level of detail needed and the specific context. “White-box” pools are most commonly named after the corresponding business process (e.g. “requirement management process”, “help-desk process” or “service delivery process”), whereas “Black-box” pools are commonly named after the corresponding organization, person or system (e.g. “supplier”, “customer” or “content management system”).

Lanes

- A Lane is a sub-partition within a pool and is used to organize and categorize activities of a process. Most commonly, a lane represents an organizational role (e.g. developer, analyst and manager). However, lanes may also be used for other purposes (e.g. first phase, second phase and third phase)



Figure 1: BPMN Swimlanes - Pool and Lanes

Activities



Task

- An atomic activity that is included within a process
- The work in the process cannot be broken down to a finer level of Process Model detail

•Application Submission



Sub-Process

- A compound activity that is included within a process.
- Can be broken down into a finer level of detail (a Process) through a set of sub-activities.

•Periodic VAT Payment



Activity Looping

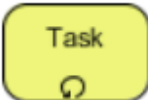
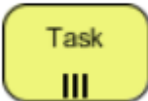
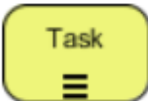
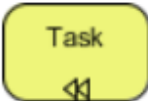
- The attributes of Tasks and Sub-Processes will determine if it is repeated or performed once.

•Product Quality Control



Task markers

You can assign markers to task. There are three markers: Loop, Multi-Instance, Compensation. A task can have one or two of these markers. Assignment of markers is done through the specification dialog box of task.

Name	Representation	Description
Loop		This marker indicates that the task will loop as long as the condition that defined in the loop is true. The condition is evaluated in each iteration, at either the beginning or the end of iteration. This marker can be used in combination with the compensation marker.
Multi-instance (parallel instances)		This marker indicates the execution of task in a desired number of instances or in a data driven approach. The instances will be started at the same time.
Multi-instance (sequential instances)		This marker indicates the execution of task in a desired number of instances or in a data driven approach. The instances will be executed one after the other.
Compensation		To undo (cancel) the result of another activity that have already been successfully completed. The execution of compensation task is due to the undesired results and possibly side effects produced by another activity that need to be reversed. A compensation task is performed by a compensation handler, which performs the steps necessary to reverse the effects of an activity.

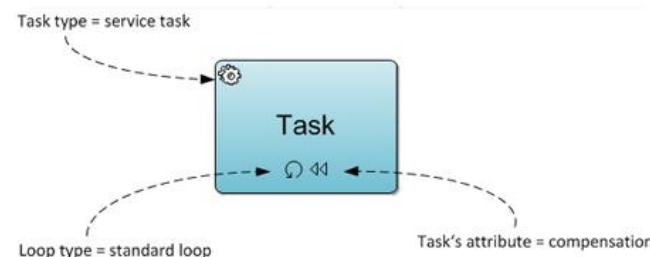
Diversity of Activity Types

A business process can include many different activity types, for example:

- Tasks, which can be performed manually, with IT systems support or completely automatically
- Tasks or sub-processes, which can perform once or repeat
- Sub-processes, which act as transactions or sub-processes with, not clearly defined sequence of tasks, etc.

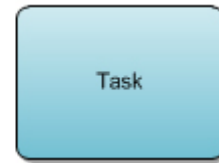
BPMN supports a diversity of different types of activities, with standardized attributes that specify their meaning and behavior. The next figure illustrates the complexity of [BPMN Activities](#) with an example of a “repeatable service-based compensation task”. Let's break the shape down to its meaning:

- The task is performed by some sort of service, which could be a web service or an automated application,
- The standard attribute defines that the task is repeated. In this case the repetition criteria also needs to be defined
- Its compensation attribute defines the task, which can only be performed when a transaction process is canceled.



Tasks vs. Sub-processes

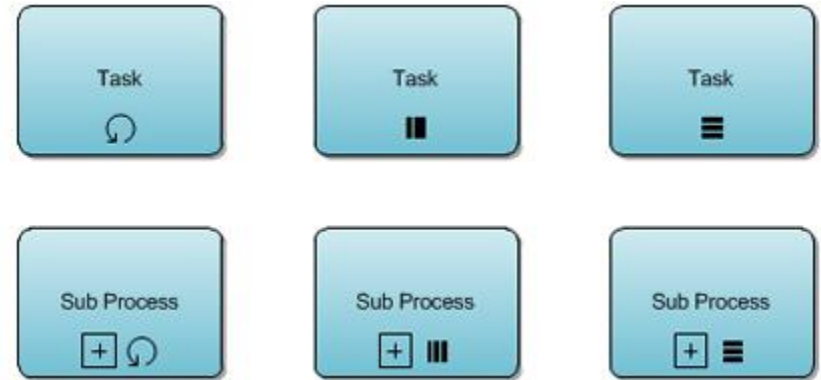
A sub-process should be only used if its details can be defined in terms of the underlying tasks or sub-processes. This means that a complex real world activity should be modeled as a task if it cannot be additionally decomposed into sub-elements, whereas a simple activity can be modeled as a sub-process if a modeler decides to additionally decompose it.



Use a sub-process if you intend to decompose it into sub-elements. Use a task if you are not planning on describing sub-elements

Loops vs. Multiple Instances

- **Use a standard loop marker** (curved arrow) when an activity has looping behavior, which means that it loops for as long as the underlying looping condition is true
- **Use a multi-instance marker** (three vertical or horizontal lines) when several activity instances are needed. This means that an Activity is performed many times with different data sets.



Events



Start Event

A Start Event indicates where a particular process will start.



Intermediate Event

Intermediate Events occur between a Start Event and an End Event. It will affect the flow of the process, but will not start or (directly) terminate the process.



End Event

An End Event indicates where a process will end.

Common Start Event Types



None

The modeler does not display the type of Event. It is also used for a Sub-Process that starts when the flow is triggered by its Parent Process.



Message

A message arrives from a participant and triggers the start of the Process.



Timer

A specific time-date or a specific cycle (e.g., every Monday at 9am) can be set that will trigger the start of the Process.



Rule

This type of event is triggered when the conditions for a rule such as "S&P 500 changes by more than 10% since opening," or "Temperature above 300C" become true.



Link

A Link is a mechanism for connecting the end (Result) of one Process to the start (Trigger) of another. Typically, these are two Sub-Processes within the same parent Process.



Multiple

This means that there are multiple ways of triggering the Process. Only one of them will be required to start the Process. The attributes of the Start Event will define which of the other types of Triggers apply.

Common Intermediate Event Types - I



None

Used to indicate some change of state in the Process



Message

A message arrives from a participant and triggers the Event. This causes the Process to continue if it was waiting for the message, or changes the flow for exception handling.



Timer

A specific time-date or a specific cycle (e.g., every Monday at 9am) can be set that will trigger the Event.



Rule

Triggered when a Rule (an expression that evaluates some Process data) becomes true.

Common End Event Types

-  None *The modeler does not display the type of Event. It is also used to show the end of a Sub-Process that ends, which causes the flow goes back to its Parent Process.*
-  Message *A message is sent to a participant at the conclusion of the Process.*
-  Terminate *All activities in the Process should be immediately ended. This includes all instances of Multi-Instances. The Process is ended without compensation or event handling.*
-  Compensation *The Compensation identifier will trigger an Intermediate Event when the Process is rolling back.*
-  Multiple *There are multiple consequences of ending the Process. All of them will occur (e.g., there might be multiple messages sent).*
-  Link *A Link is a mechanism for connecting the end (Result) of one Process to the start (Trigger) of another. Typically, these are two Sub-Processes within the same parent Process. A Token arriving at Link End Event will immediately jump to its corresponding target Start or Intermediate Event.*

Gateway Types

Exclusive gateway: Represents a decision point where only one of the outgoing paths can be taken.

Inclusive gateway: Represents a decision point where multiple paths can be taken based on evaluating conditions associated with each outgoing sequence flow.

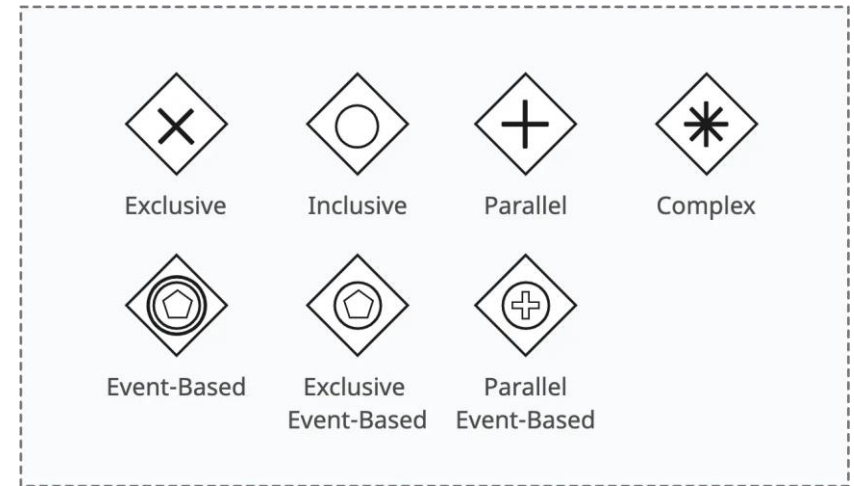
Parallel gateway: Represents a point where multiple paths can be taken simultaneously without evaluating conditions.

Complex gateway: Represents a more complex decision point where conditions and rules may involve a combination of logical operators.

Event-based gateway: Represents a decision point based on events. It is used when the process flow depends on the occurrence of specific events, such as receiving a message or a timer event.

Exclusive event-based gateway: Similar to the event-based gateway, but enforces exclusive decision-making based on events. Only one path can be taken, depending on the first event to occur.

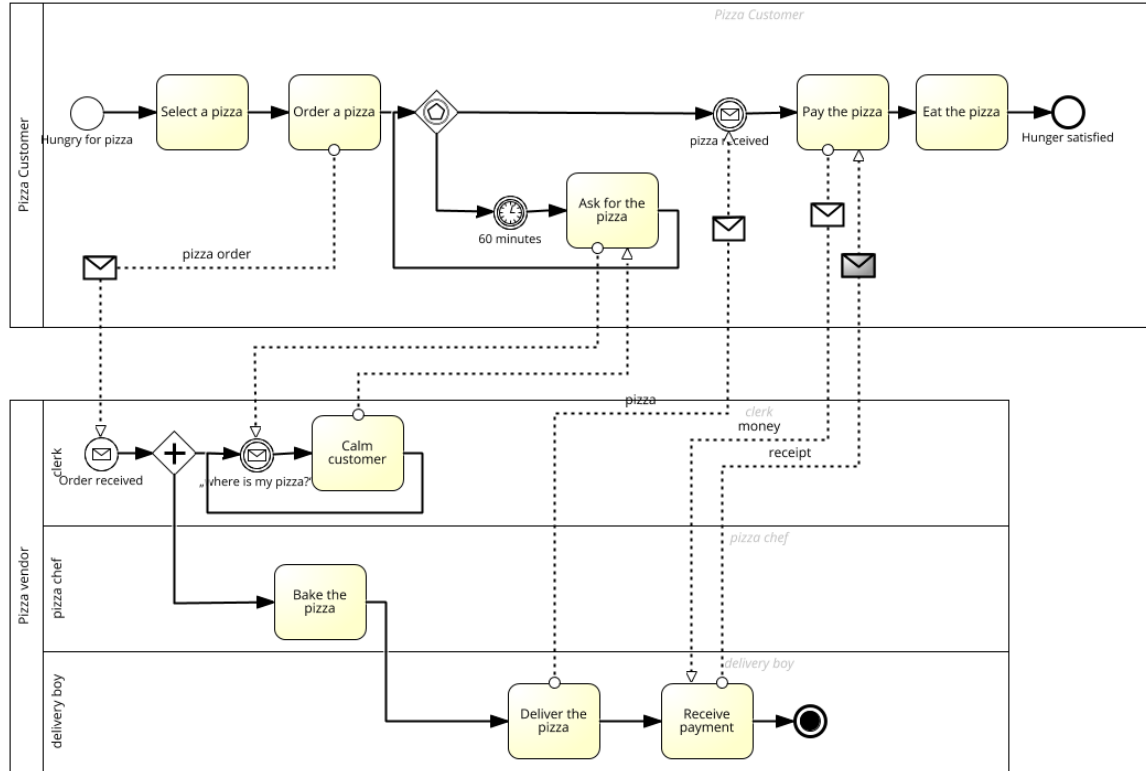
Parallel event-based gateway: The occurrence of all subsequent events starts a new process instance.



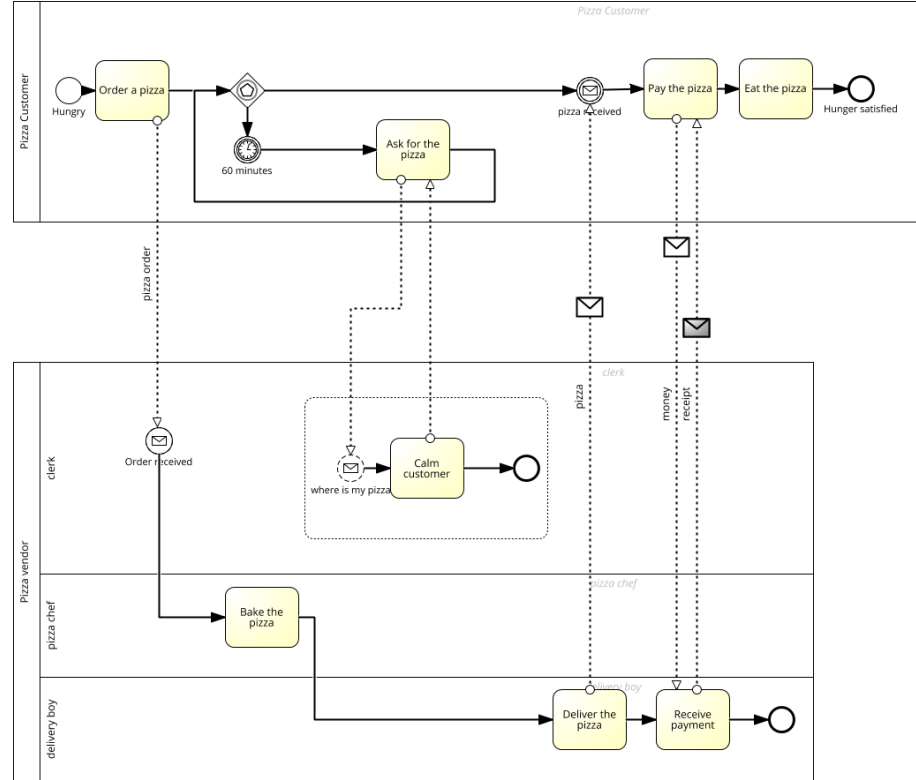
BPMN Example #1 - Pizza



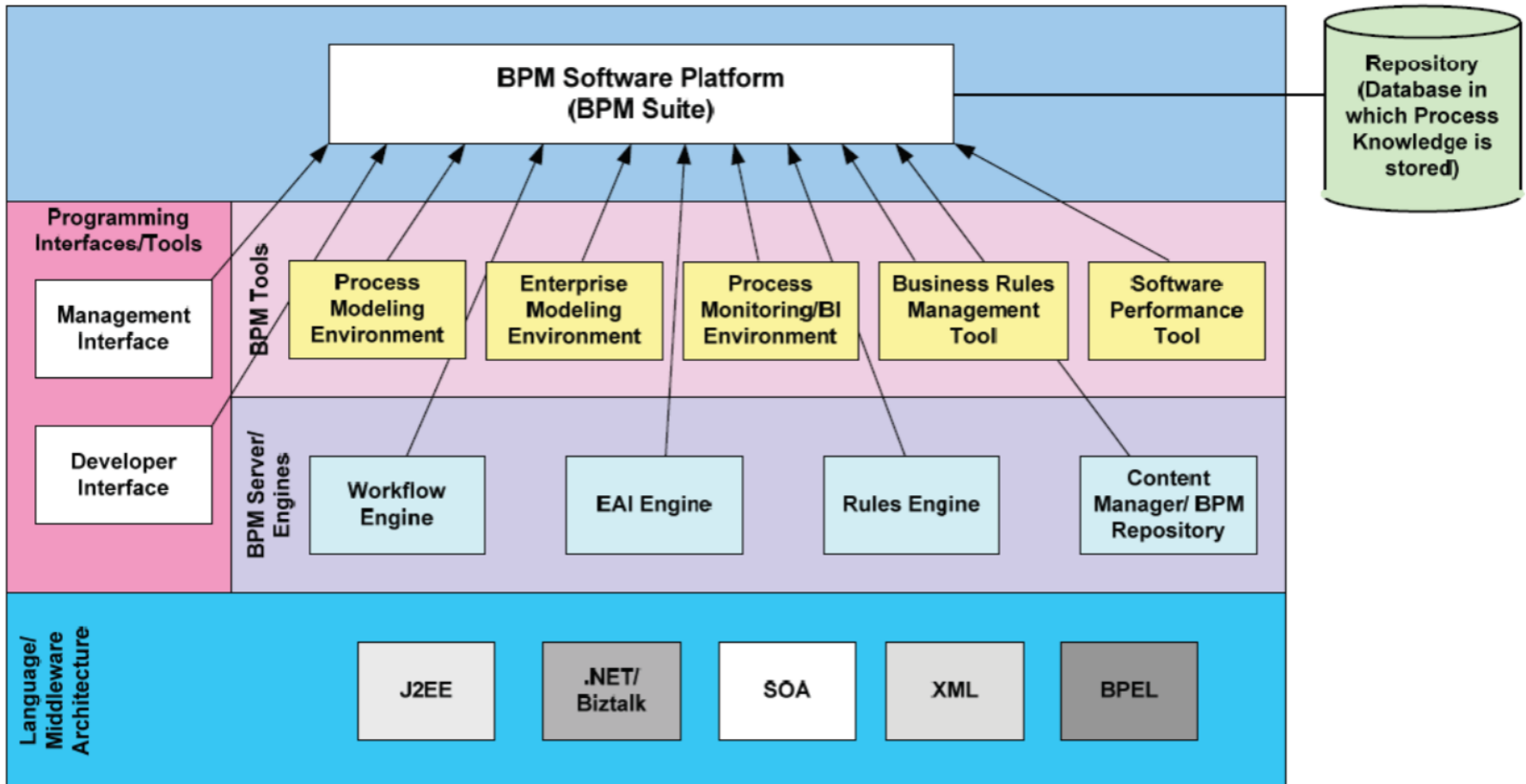
Pizza Delivery



Pizza Delivery (Event Sub-process)



General Architecture of BPM Platforms



Criteria to select a Business Process Modelling Tool

- Architecture (1-tier, client/server, n-tier)
- Scalability by inserting new components and / or libraries
- Support for a series of methodologies and notations
- Availability of a Business Process Models Library
- Potential to automatically translate a business process to code
- Integration with other business process tools
- Unified management of models created by multiple users
- Existence of a web user interface
- Users management, privacy and access rights
- Provision for multilingual content
- Maturity
- Technical support and frequent new versions
- Licensing scheme and cost

Camunda in 2026: Camunda 8, and the end of Camunda 7

- Camunda 8 is a process orchestration platform. You draw the process in BPMN, deploy it, and the Zeebe engine runs each instance: tasks, timers, messages, decisions.
- Components: Zeebe (engine), Operate (see running instances), Tasklist (human tasks), Optimize (analytics), Web and Desktop Modeler, Connectors (prebuilt tasks for REST, Slack, AWS and more), Identity.
- Your services are workers. A worker subscribes to a task type, receives jobs over gRPC or REST, does the work and completes the job. Any language.
- Run it as SaaS on camunda.io, which tonight's exercise uses, or self-managed on Kubernetes with the Helm charts.
- Releases every April and October. 8.9, April 14, 2026, added the Agent2Agent protocol for AI agents and relational databases (PostgreSQL, MySQL and others) as secondary storage beside Elasticsearch.
- Camunda 7, the embedded Java engine many companies still run: Community Edition ended with 7.24 on October 14, 2025. Enterprise Edition patches continue to April 2030, extended support to 2032. New work goes on Camunda 8.

BREAK

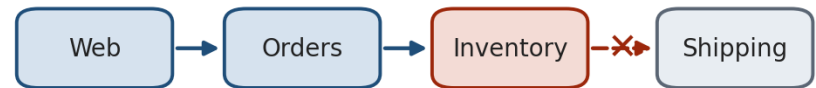
Queues, topics, delivery guarantees, and the patterns that make them safe to build on

PART III: MESSAGING AND EVENT-DRIVEN INTEGRATION

Why not just call the next service?

- A chain of synchronous calls fails together. The slowest hop sets the response time for the whole request.
- A queue between two services decouples them in time. The producer finishes when the message is stored, not when the work is done.
- The queue absorbs bursts. The consumer drains at its own pace, and a consumer that is down catches up when it returns.
- The cost: the caller no longer knows the outcome. The result arrives later, as another message or a status the caller can query.
- Rule of thumb: if the caller needs the answer to continue, call. If the caller needs the work done eventually, queue.

Synchronous call chain



Inventory is slow, so Orders waits, so Web times out.
One slow hop fails the whole request.

Queue in between

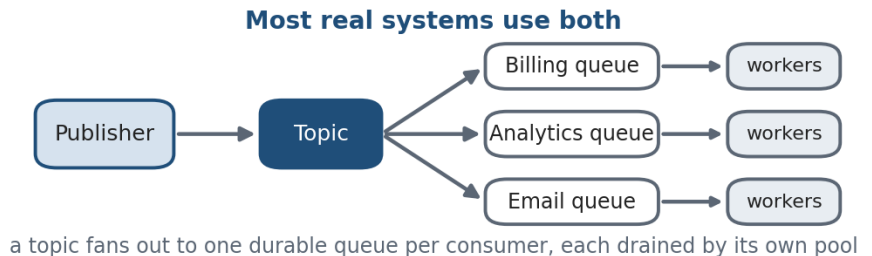
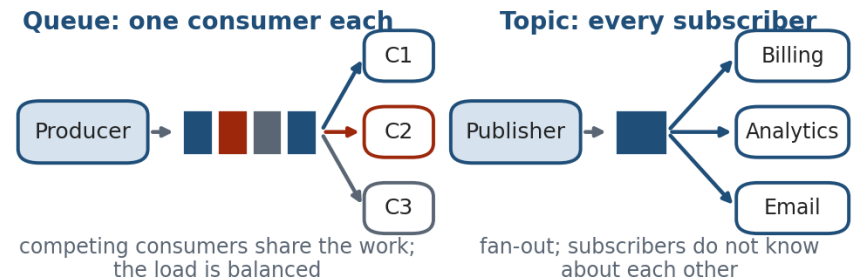
Orders writes the message and answers the user at once.
Inventory catches up when it can.



queue holds the backlog

Queue or topic

- Queue, point to point: each message is delivered to one consumer. Several consumers on one queue share the work.
- Topic, publish and subscribe: each message is delivered to every subscriber. The publisher does not know who listens.
- Most systems use both. A topic fans out to one durable queue per consuming service; each service drains its own queue with a pool of workers.
- Ask two questions about every message: how many kinds of consumer need it, and does a consumer need to keep it if it is down?

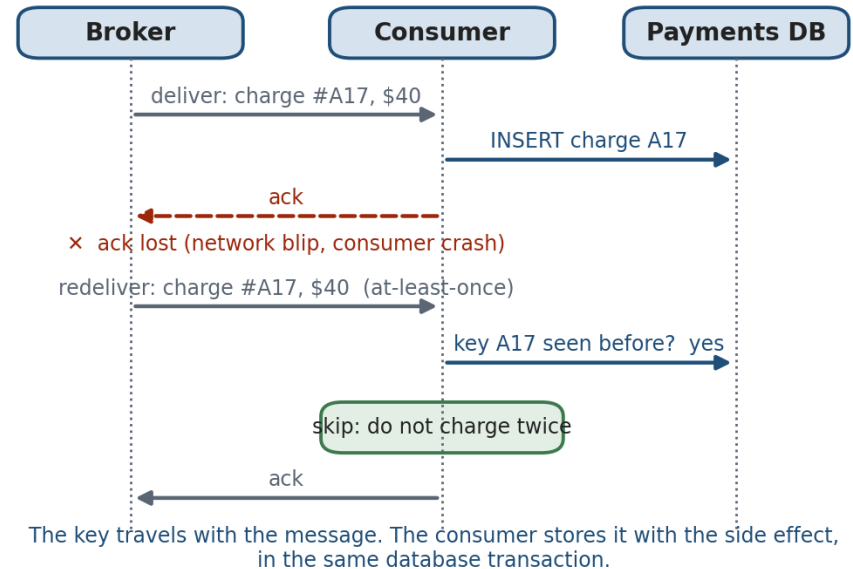


Delivery guarantees: pick one, and design for it

- At most once: the broker sends and forgets. Messages can be lost. Fine for metrics samples, wrong for orders.
- At least once: the broker redelivers until the consumer acknowledges. Messages can arrive twice. This is what SQS standard queues, SNS and Kafka give you by default.
- Exactly once: no message lost, none duplicated. Between two systems this is a contract you build, not a setting you turn on. Kafka's transactions and SQS FIFO give exactly-once processing inside their own boundaries.
- So the working assumption for every consumer you write is: at least once. Duplicates are normal traffic, not an incident.
- Acknowledge after the side effect is committed, never before. An ack before the commit turns at-least-once back into at-most-once.

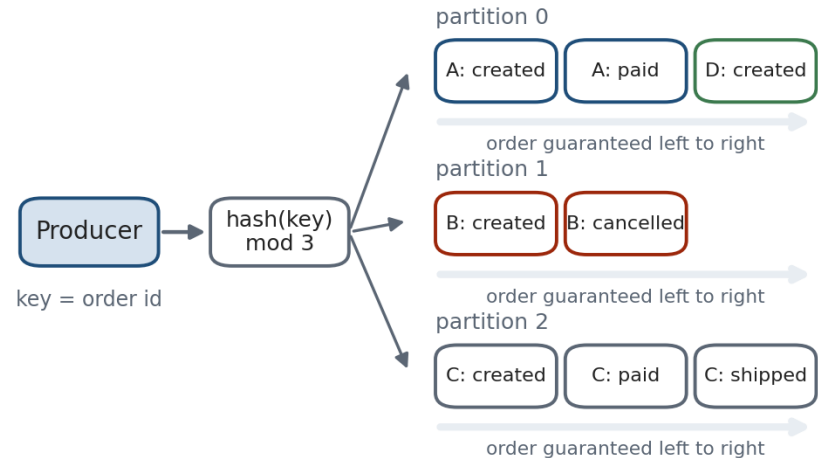
Duplicates will happen: idempotency keys

- Idempotent: doing it twice has the same effect as doing it once.
- Every message carries a key: an order id, a payment id, or a UUID the producer generates.
- The consumer records the key in the same database transaction as the side effect. On redelivery it finds the key and skips.
- Write handlers as set-to-a-value or upsert, not increment. SET balance = 60 is safe to repeat; balance = balance + 20 is not.
- The key also goes in the log line. When a customer asks why they were charged twice, the key is how you find out that they were not.



Ordering: only within a key

- A broker scales by splitting a topic into partitions. Order is guaranteed inside a partition, not across them.
- Messages with the same key go to the same partition. Choose the key so that everything that must stay in sequence shares it: the order id, the account id, the device id.
- SQS FIFO queues do the same with a message group id. SQS standard queues make no ordering promise at all.
- A hot key is a hot partition. If one customer generates half the traffic, one consumer does half the work.
- Design so that out-of-order across keys does not matter. If it does, your key is wrong.

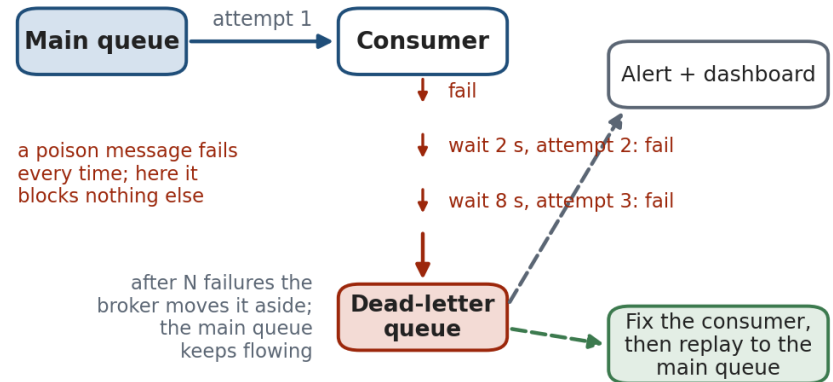


Same key, same partition, so one order's events stay in sequence.
Across keys there is no order, and there does not need to be.

Retries, backoff and the dead-letter queue

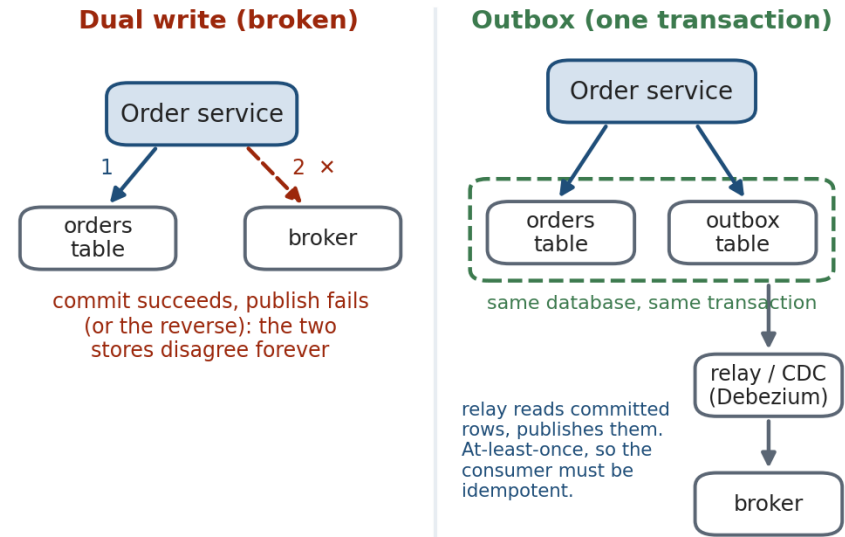
- A consumer that fails releases the message, and the broker delivers it again after a delay.
- Retry with exponential backoff and jitter: 2 s, 4 s, 8 s, plus a random offset. A fixed one-second retry from a thousand workers is an outage generator.
- After N failures the broker moves the message to a dead-letter queue. The main queue keeps flowing; the bad message waits for a person.
- Alert on dead-letter queue depth. Nobody looks at a queue that nobody is alerted on.
- Fix the consumer, then replay the dead-letter queue back into the main one. Because the consumer is idempotent, replaying is safe.

Exponential backoff with jitter keeps a sick downstream from being hammered.



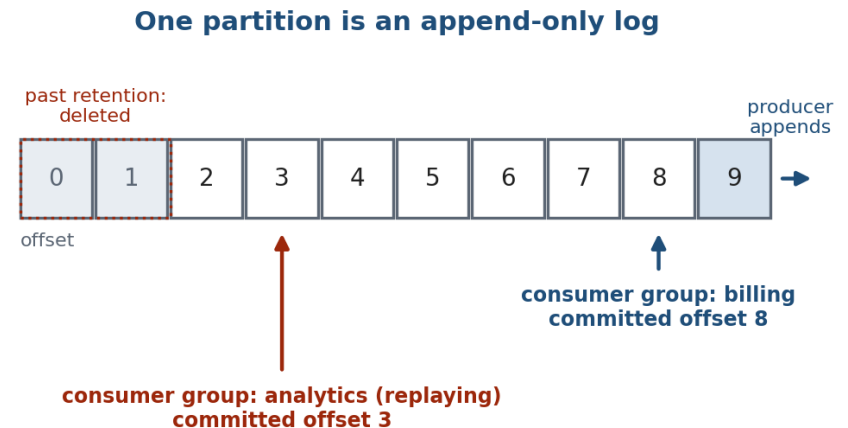
The dual-write problem and the outbox pattern

- A service commits a row to its database and then publishes an event. If the publish fails, the databases disagree forever. If the commit fails after the publish, the event lies.
- There is no distributed transaction across a database and a broker that you would want to run in production.
- Outbox: write the event into an outbox table in the same local transaction as the business row. Either both happen or neither does.
- A relay publishes outbox rows to the broker. Polling works; change data capture, for example Debezium reading the database log, is the usual production choice.
- The relay is at-least-once, so the consumers must be idempotent. Everything on this slide depends on the previous three.



Apache Kafka in one slide

- A topic is a set of partitions. A partition is an append-only log on disk. Each message has an offset.
- Consumers do not remove messages. Each consumer group tracks its own committed offset, so billing and analytics read the same log independently.
- Retention is by time or size, not by consumption. Seven days is a common default; a compacted topic keeps the latest value per key forever.
- Replay is built in: start a new consumer at offset 0 and rebuild its state from history.
- Kafka 4.x runs without ZooKeeper; 4.3 is the current release, May 2026. Managed offerings: Confluent Cloud, now part of IBM; Amazon MSK; Azure Event Hubs on the Kafka protocol.



Each group keeps its own position. Reading does not remove anything.
A new consumer can start from offset 0 and rebuild its state from history.

SQS, SNS and EventBridge: which one for what

- SQS is a queue. Standard queues: at-least-once, best-effort ordering, nearly unlimited throughput. FIFO queues: ordered and exactly-once processing within a message group, at lower throughput per group. Visibility timeout, dead-letter queues, long polling.
- SNS is publish and subscribe. A topic fans out to SQS queues, Lambda functions, HTTP endpoints, email and SMS. Subscription filters route by message attributes.
- EventBridge is an event bus with rules. Rules match on the content of the event and route to AWS services, other accounts and SaaS partners. It has a schema registry and archive-and-replay.
- Choose: SQS to buffer work for one consumer. SNS to fan one message out to several. EventBridge to route by content across teams, accounts and vendors.
- The SQS payload limit rose to 1 MiB in August 2025 and SNS matched it on September 18, 2026. A message that large is still usually a design smell; send a reference, not the document.

Events carry schemas

- A message without an agreed schema is a coupling you cannot see. The producer changes a field name and three consumers break at 2 AM.
- Serialize with a schema: Avro, Protocol Buffers or JSON Schema. Register each version in a schema registry: Confluent Schema Registry, AWS Glue Schema Registry, the EventBridge schema registry.
- The registry enforces a compatibility rule. Backward: new readers read old messages. Forward: old readers read new messages. Full: both.
- Practical rules: add optional fields, never rename or retype one, never reuse a field number, and put the event type and version in the envelope.
- Treat the schema as an API. Review changes the way you review an endpoint change, with the consumers in the room.

Commands, events and change data capture

- A command asks one recipient to do something, and can be refused: ReserveStock. An event states that something happened, in the past tense, and cannot be refused: StockReserved.
- Commands go to queues. Events go to topics. Mixing them up produces a topic where every subscriber thinks it is the one responsible.
- Event notification carries an id and little else; consumers call back for detail. Event-carried state transfer carries the data; consumers keep their own copy and never call back. The second scales; the first stays consistent.
- Change data capture turns a database's own commit log into events, row by row. Debezium is the common open-source tool. It is how legacy systems join an event-driven architecture without being rewritten.
- Event sourcing goes one step further and makes the event log the system of record. Powerful, and a commitment; do not adopt it by accident.

The idempotent consumer: build this every time

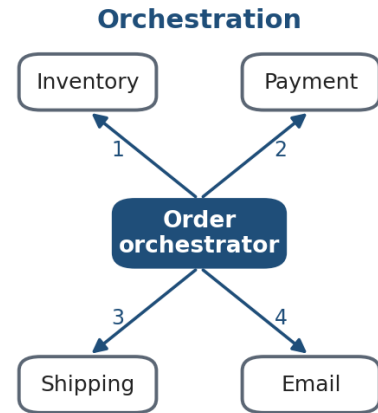
- Every message has a key, set by the producer.
- Store the processed key in the same transaction as the side effect.
- Handlers set values; they do not increment them.
- Acknowledge after the commit, never before.
- Retries back off with jitter and stop after N; the rest goes to a dead-letter queue that someone is paged on.
- The key and a correlation id are in every log line.
- Consumer lag and dead-letter depth are on a dashboard.
- Replaying the last day of messages against a copy of the database changes nothing. If it does, you are not finished.

Orchestration, choreography, sagas, and the engines that run them

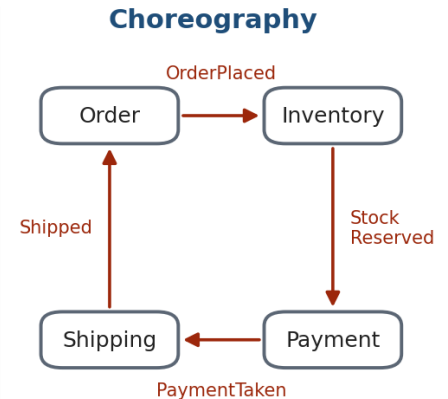
PART IV: SERVICE COMPOSITION

Orchestration or choreography

- Orchestration: one component holds the flow and calls each service in turn. The process is explicit, readable and testable in one place. The orchestrator is a single point to protect, and it can grow into a new monolith.
- Choreography: each service reacts to events and emits its own. No central flow, and adding a listener touches nothing else. Nobody can read the end-to-end process, and a missing event is silent.
- Use orchestration when the flow has an owner, a deadline, or a person in it. Use choreography for side effects that the main flow should not wait for: analytics, notifications, search indexing.
- Most real systems orchestrate the core transaction and choreograph the edges.



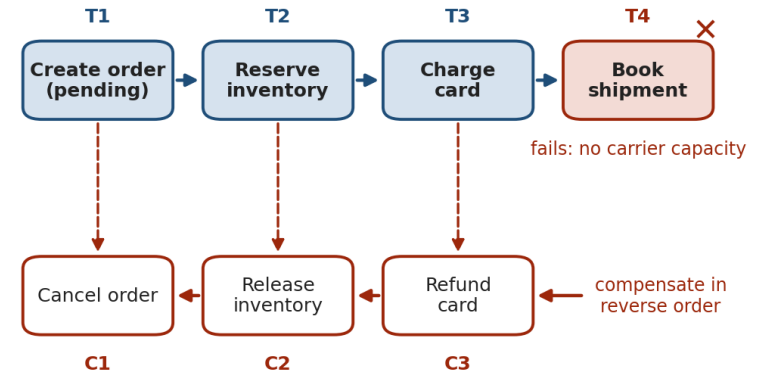
one component knows the whole flow. Explicit, readable, testable. One point to protect.



each service reacts to events and emits its own. Easy to add a listener. The flow is written nowhere.

Sagas: one business transaction, many local ones

- A saga is a sequence of local transactions, one per service. Each commits on its own. There is no rollback across services.
- When a step fails, the saga runs compensating actions for the steps that already committed, in reverse order. A refund is a new transaction, not an undo.
- Compensation is a business decision. Cancel the order? Refund minus a fee? Ship anyway and apologise? Decide before you write the step.
- Some steps cannot be compensated: an email sent, a payment settled. Put those last, or make them the pivot step after which nothing can fail.
- Orchestrated sagas keep the state in the orchestrator. Choreographed sagas keep it in the events. Either way, the saga needs an id and a timeout.



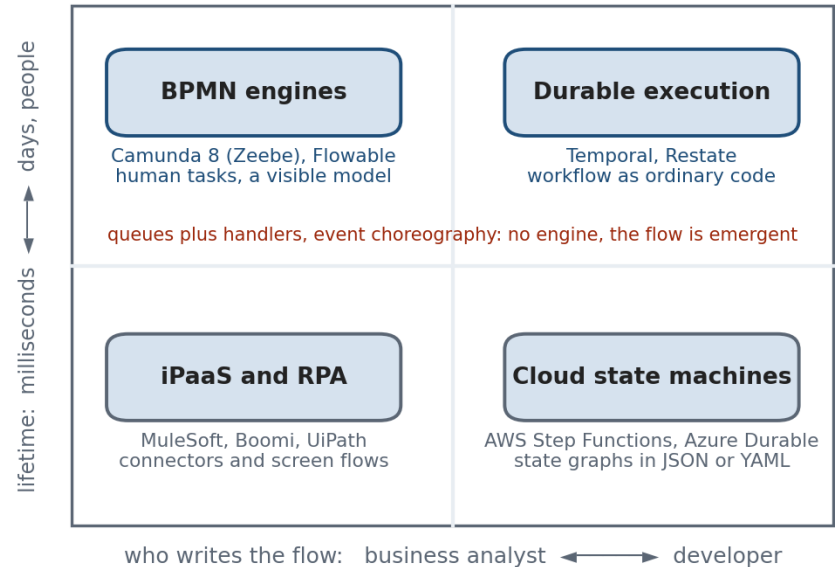
Each step commits locally. There is no rollback across services, only a compensating action that undoes the business effect. The refund is a new transaction, not an undo.

Three workflow engines

- Camunda 8. The flow is a BPMN model; Zeebe executes it. Human tasks, timers, message correlation and DMN decision tables are first class. Workers in any language pick up jobs over gRPC or REST. SaaS on camunda.io or self-managed on Kubernetes.
- Temporal. The flow is ordinary code in Go, Java, Python, TypeScript or .NET. The server records every step in an event history and replays it after a crash, so a workflow can sleep for a month and resume. No diagram; the code is the model. Temporal Cloud or self-hosted.
- AWS Step Functions. The flow is a JSON state machine. Serverless, priced per state transition, and wired to more than two hundred AWS services without code. Weak outside AWS.
- Also in this space: Azure Durable Functions and Logic Apps, Google Cloud Workflows, Restate, Flowable, and Netflix Conductor.
- All of them give you the same three things: durable state between steps, retries with policy, and a place to see where every instance is.

Where a BPMN engine fits, and where it does not

- Two axes: who writes the flow, and how long an instance lives.
- BPMN engines win when a business analyst must read and change the flow and when people act inside it: approvals, claims, onboarding, anything measured in days.
- Durable-execution engines win when developers own the flow and it is mostly machine to machine: order pipelines, provisioning, data jobs, AI agent steps.
- Cloud state machines win for short serverless glue inside one cloud.
- No engine at all is right for pure fan-out. Do not put a workflow engine in front of a topic.



Composition pitfalls

- The distributed monolith: services that call each other synchronously and share a database. All the cost of microservices, none of the independence.
- Stacked retries. Three layers that each retry three times make twenty-seven attempts at the bottom. Retry at one layer, and set the deadline at the top.
- Chatty flows. Twelve calls to render one page. Compose at the edge or carry the state in the event.
- No correlation id. Without one id that travels through every message and log line, an incident is archaeology.
- The orchestrator that knows everything. Business rules migrate into it until it is the monolith you left.
- Choreography nobody can trace. If the question where is order 123 has no answer, add an orchestrator or a process-tracking view.
- Consumers coupled to a producer's internal tables through change data capture. Publish a public event schema, not your database.

Choosing for your project: a short decision list

- Does a person have to act in the middle of the flow? Then a workflow engine with human tasks. Camunda is tonight's example.
- Is it a few services, owned by developers, machine to machine? Then Temporal or Step Functions, or a plain queue with an idempotent consumer.
- Is the job only telling other services that something happened? Then a topic and queues, no engine.
- Is any step irreversible? Design its compensation, or make it the last step, before you write it.
- Can you replay yesterday's messages without damage? If not, stop adding features and fix the consumer.
- Write the answers into your project abstract's architecture section. The midterm asks these questions about a scenario.

In-class exercise 1: Amazon SQS

- Getting started with Amazon SQS. Manage queues and messages using the Amazon SQS console.
- Prerequisites
- Step 1: Create a queue
- Step 2: Send a message
- Step 3: Receive and delete your message
- Step 4: Delete your queue
- <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-getting-started.html>
- Submit one document per project team (doc, docx or pdf) with a screenshot of each step, through In-Class: Amazon SQS on Canvas. Section 49: due tonight, Wed Sep 30, 11:59 PM. Section 03: due Tue Oct 6, 11:59 PM.

In-class exercise 2: Camunda



1. [Create a Camunda 8 account](#) - Create a Camunda 8 account to create clusters, deploy processes, and create a new instance.
2. [Model your first process](#) - Design and deploy a process, and share the process with your teammates or other stakeholders to begin collaborating.
3. [Create a cluster](#) - Create your first cluster to execute processes.
4. [Orchestrate human tasks](#) - Assign human tasks to users so they can enter the necessary data to drive the business process forward.
5. [Get started with API orchestration](#) - Configure Connector tasks with domain-specific parameters.
6. [Orchestrate microservices](#) - Orchestrate the microservices necessary to achieve your end-to-end automated business process.

Submit one document per project team (doc, docx or pdf) with screenshots, through In-Class: Camunda on Canvas. Section 49: due tonight, Wed Sep 30, 11:59 PM. Section 03: due Tue Oct 6, 11:59 PM.

Project Abstract Submission: due tonight

- Project Title/Name
- Abstract (300-500 words)
- Project Milestones (Including Dates)
- Project Requirements, Equipment, Risks
- Submitted by Team Leader only (one per team)
- Section 49: due tonight, Wed Sep 30, 11:59 PM, through Project Abstract Submission on Canvas. Abstract presentations and the midterm are next session, Wed Oct 7.
- Section 03: due Tue Oct 6, 11:59 PM. Presentations and the midterm on Tue Oct 13.
- Use tonight's vocabulary in the architecture part: which calls are synchronous, which go through a queue or topic, and where the flow has an owner.

References

- OMG, Business Process Model and Notation (BPMN) 2.0 specification. <https://www.omg.org/spec/BPMN/2.0>
- Camunda 8 documentation. <https://docs.camunda.io>
- Apache Kafka documentation. <https://kafka.apache.org/documentation>
- AWS decision guide: Amazon SQS, Amazon SNS, or Amazon EventBridge? <https://docs.aws.amazon.com/decision-guides/latest/decision-guides/sns-or-sqs-or-eventbridge.html>
- Chris Richardson, microservices.io patterns: Saga, Transactional outbox, Idempotent consumer. <https://microservices.io/patterns>
- Gregor Hohpe and Bobby Woolf, Enterprise Integration Patterns, 2003. The vocabulary every broker still uses.
- Martin Kleppmann, Designing Data-Intensive Applications, 2017, chapter 11, Stream Processing.
- Martin Fowler, What do you mean by Event-Driven?, 2017. <https://martinfowler.com/articles/201701-event-driven.html>
- Sam Newman, Building Microservices, 2nd edition, 2021, chapters 4 and 6.
- Temporal documentation. <https://docs.temporal.io>

Next session

- Section 49: Wednesday, October 7. Project abstract presentations, then the midterm exam, in class.
- Section 03: Tuesday, October 13. Same session.
- The midterm covers sessions 1 to 7. Tonight's material is on it: queue against topic, at-least-once and idempotency, the outbox pattern, orchestration against choreography, sagas.
- Format and rules for the presentations and the exam are in the syllabus and on Canvas.
- Before then: the two in-class exercises and the project abstract, due tonight for section 49.

SAN JOSÉ STATE UNIVERSITY *powering* SILICON VALLEY